ARQUITETURA DE DESENVOLVIMENTO COM IA

OpenCode CLI + Ollama · Orquestração híbrida · Fallback automático

Projeto de referência para notebook Dell Inspiron, Intel Core i7 de 13ª geração, vídeo integrado e 16 GB de RAM

Objetivo

Definir uma arquitetura de desenvolvimento por terminal, sem dependência de IDE, que distribua tarefas entre agentes especializados, priorize modelos locais para reduzir consumo de tokens pagos e continue trabalhando com IA local quando o provedor pago atingir limites ou ficar indisponível.

Princípio central: vários agentes podem compartilhar um único modelo local. Agentes são perfis de trabalho, não necessariamente modelos independentes. Para 16 GB de RAM, a concorrência precisa ser controlada.

Documento de arquitetura e plano de implementação · Outubro de 2026

Visão geral

A solução combina uma interface CLI, um orquestrador de tarefas, agentes especializados, um motor local de inferência e ferramentas determinísticas de desenvolvimento. A nuvem é utilizada quando disponível e justificável; a execução local é o caminho de continuidade quando a cota paga acaba.

CAMADA	COMPONENTES	RESPONSABILIDADE
1. Interface	OpenCode CLI, terminal, Git	Receber pedidos, inspecionar arquivos, editar e executar comandos autorizados.
2. Orquestração	Agente principal	Planejar, classificar complexidade, delegar, acompanhar e consolidar resultados.
3. Agentes	Explorer, Laravel, testes, revisão, docs, segurança, arquitetura	Executar subtarefas delimitadas com contexto mínimo e permissões apropriadas.
4. Inferência	Ollama + modelos locais; provedor pago	Gerar respostas e código. Alternar para local em caso de cota ou indisponibilidade.
5. Ferramentas	PHP, Composer, PHPUnit/Pest, Git, banco de testes	Executar operações determinísticas sem gastar tokens de IA.

Fluxo lógico de execução

Pedido no terminal
OpenCode CLI

↓

Orquestrador
planeja e identifica dependências

↓

Roteamento
local para tarefas simples; pago para tarefas complexas quando disponível

↓

Fallback
quota / rate limit / indisponibilidade → agente local

↓

Validação
testes, diff, revisão e relatório final

1. Dimensionamento para o notebook

Equipamento considerado: Dell Inspiron, Intel Core i7 de 13ª geração, GPU integrada e 16 GB de RAM. A GPU integrada compartilha memória com o sistema; portanto, o limite real depende do modelo exato do processador, do sistema operacional, do contexto e da quantização.

ITEM	PONTO DE PARTIDA	OBSERVAÇÃO
Modelos carregados	1 modelo local por vez	Evita duplicar o consumo de RAM.
Concorrência de inferência	1 chamada local simultânea	Aumentar para 2 só após medir memória e latência.
Agentes configurados	6 a 8 perfis	Não significa 6 a 8 modelos em execução.
Contexto inicial	4K–8K tokens	Contexto maior eleva o uso de memória; ajuste por tarefa.
Sistema operacional	Ubuntu, Windows ou WSL2	A configuração final deve corresponder à versão real do OpenCode.

Modelos locais para avaliar

MODELO	USO RECOMENDADO	CUIDADOS
Qwen2.5-Coder 1.5B Instruct	Tarefas simples, rascunhos e classificação leve.	Mais rápido e limitado; validar qualidade.
Qwen2.5-Coder 3B Instruct	Primeiro modelo para exploração e tarefas rotineiras.	Bom ponto inicial para avaliar desempenho local.
Qwen2.5-Coder 7B Instruct, Q4	Implementações pequenas e refatorações delimitadas.	Maior pressão de memória e latência; testar antes de adotar.

A quantização Q4 reduz o tamanho do modelo, mas não determina sozinha o uso total de RAM. O contexto, o cache KV, o backend e os demais programas também contam. Não planeje executar vários modelos médios simultaneamente nesse equipamento sem benchmark.

Comandos iniciais

```
ollama pull qwen2.5-coder:3b
ollama run qwen2.5-coder:3b
# Opcional, após medir recursos:
ollama pull qwen2.5-coder:7b
```

2. Catálogo de agentes

Os perfis abaixo podem compartilhar o mesmo modelo local. Os agentes pagos são usados quando disponíveis e quando a tarefa exige maior capacidade; a configuração deve indicar explicitamente os modelos permitidos para cada perfil.

AGENTE	MODALIDADE	RESPONSABILIDADE E LIMITES
orquestrador	Pago preferencial; local como contingência	Planejar, rotear, controlar tentativas e consolidar resultados. Não reler todo o repositório sem necessidade.
explorer-local	Local 3B	Somente leitura: encontrar arquivos, classes, métodos, rotas e testes relevantes; devolver caminhos e resumo.
laravel-local	Local 3B/7B	Alterações pequenas e isoladas em Laravel; seguir convenções do projeto e limitar o diff.
testes-local	Local 3B ou ferramentas determinísticas	Selecionar e executar testes; interpretar falhas; não declarar sucesso sem saída verificável.
review-local	Local 3B	Revisar diff em busca de regressões e inconsistências; revisão preliminar, não certificação de segurança.
docs-local	Local 3B	Criar documentação e resumos objetivos a partir dos arquivos relevantes.
security-review	Pago quando disponível; local como apoio	Analisar autenticação, autorização, exposição de dados e isolamento multi-tenant; revisão crítica exige validação adicional.
architect	Pago quando disponível; local em contingência	Avaliar decisões de arquitetura e alterações que cruzam muitos módulos.

Estrutura sugerida de arquivos

```
.opencode/
■■■ agents/
    ■■■ explorer-local.md
    ■■■ laravel-local.md
    ■■■ testes-local.md
    ■■■ review-local.md
    ■■■ docs-local.md
```

A sintaxe e os campos exatos dependem da versão do OpenCode. Antes de aplicar, confirme o schema da versão instalada e valide os identificadores dos modelos disponíveis no provedor Ollama. Não misture configurações de versões diferentes.

3. Política de roteamento e fallback

A regra principal é preservar a continuidade da tarefa quando o provedor pago não puder atender. Depois de detectar esgotamento de cota, o sistema deve manter a tarefa no modo local em vez de insistir repetidamente no provedor.

Tabela de decisão

SITUAÇÃO	AÇÃO	CONTROLE
Leitura, busca e resumo de código	Agente local de exploração	Contexto mínimo; somente arquivos relevantes.
Teste determinístico disponível	Executar comando local	Não gastar chamadas de IA para executar comandos previsíveis.
Implementação pequena e isolada	Agente local de implementação	Diff limitado; rodar testes pertinentes.
Arquitetura ou mudança transversal	Modelo pago se disponível	Se falhar por cota, decompor e continuar localmente.
Quota esgotada / rate limit persistente	Ativar fallback local	Evitar retries repetidos; manter modo local até fim da tarefa.
Erro de autenticação/configuração	Registrar a causa e tentar local	Fallback não corrige credenciais ou configuração do provedor.
Modelo local falha	Tentar alternativa local configurada	Se não houver alternativa, preservar progresso e informar bloqueio.

Pseudofluxo de controle

```
receber tarefa
  -> classificar escopo e risco
  -> coletar evidências com ferramentas locais
  -> se tarefa simples: executar com agente local
  -> se tarefa complexa e provedor pago disponível: usar modelo pago
  -> se quota/rate limit/indisponibilidade:
       registrar falha
       parar retries pagos repetidos
       preservar plano e resultados
       mudar para agente local
       dividir tarefa em etapas menores
  -> executar testes
  -> revisar diff e resultado
  -> informar modalidade, alterações, testes e pendências
```

Importante: um prompt por si só não garante fallback. O mecanismo precisa tratar o erro de provedor fora da resposta do modelo ou usar um recurso de fallback suportado pela versão instalada. A implementação exata depende das capacidades de configuração/plugins disponíveis; não se deve presumir que um erro de API será sempre interpretado pelo agente.

4. Economia de tokens e controle de contexto

A economia vem de reduzir contexto desnecessário, evitar trabalho duplicado e reservar chamadas pagas para tarefas que realmente precisam delas. Mais agentes não significam automaticamente menor custo.

- Busca antes de leitura: use grep, ripgrep, find, Git e ferramentas de navegação para localizar arquivos antes de abrir conteúdo extenso.
- Resumos curtos: agentes exploradores devolvem caminhos, linhas relevantes, evidências e dúvidas — não cópias completas de arquivos.
- Escopo explícito: cada agente recebe uma tarefa, arquivos-alvo, restrições e critério de conclusão.
- Evitar ciclos: limitar tentativas de reparo e proibir alternância repetida entre modelo pago e local após quota esgotada.
- Testes sem IA: usar Composer, PHPUnit/Pest, linters, formatters e comandos do projeto diretamente no terminal.
- Cache de contexto: não reexplicar o projeto em toda chamada; manter convenções e comandos em arquivos de instruções versionados.
- Paralelismo com cuidado: executar em paralelo apenas tarefas independentes e limitar a concorrência conforme memória e latência.

Métricas para acompanhar

MÉTRICA	POR QUE MEDIR
Percentual de tarefas concluídas localmente	Indica quanto trabalho deixou de depender de modelos pagos.
Chamadas pagas por tarefa	Ajuda a encontrar etapas que usam o modelo caro desnecessariamente.
Tempo por tarefa e por modelo	Compara custo de latência e produtividade real.
Taxa de sucesso dos testes	Evita otimizar custo sacrificando a qualidade.
Retries e falhas de fallback	Revela ciclos de erro, configuração incorreta ou modelo local insuficiente.

5. Segurança e integridade do código

- Agente de exploração deve ser somente leitura; conceda escrita apenas aos agentes que implementam mudanças.
- Não executar comandos destrutivos, migrações em produção ou operações irreversíveis sem autorização.
- Preservar alterações existentes e verificar o estado do Git antes e depois da tarefa.
- Nunca incluir segredos, tokens de acesso ou arquivos de ambiente em prompts desnecessários.
- Para Laravel, validar policies, gates, validação de entrada, transações, tenancy e escopo de consultas.
- Em tarefas que alterem banco de dados, pagamentos, permissões ou isolamento de clientes, exigir testes específicos e revisão reforçada.
- O modelo local pode ser menos confiável em tarefas complexas; fallback significa continuidade, não garantia de equivalência com o modelo pago.

Exemplo de instrução para o agente explorador

```
---
description: Encontra evidências relevantes no repositório
mode: subagent
model: ollama/qwen2.5-coder:3b
---

Você é um agente de exploração em modo somente leitura.
- Localize arquivos, classes, métodos, rotas e testes relevantes.
- Não altere arquivos nem execute comandos destrutivos.
- Leia apenas o contexto necessário.
- Retorne caminhos e referências objetivas.
- Não reproduza arquivos inteiros.
- Se não houver evidência suficiente, declare a incerteza.
```

Este bloco é um exemplo de prompt e estrutura, não uma configuração universal garantida. Confirme o identificador de modelo e o schema suportado pela versão instalada.

6. Plano de implementação

Fase 1 — Base funcional

- Confirmar sistema operacional, versão do OpenCode e versão do Ollama.
- Validar o endpoint local e executar um teste simples com um modelo pequeno.
- Registrar memória disponível, latência e velocidade em tarefas reais.

Fase 2 — Agentes locais

- Criar explorer-local, laravel-local, testes-local, review-local e docs-local.
- Definir permissões mínimas e limites de contexto.
- Testar cada agente individualmente antes de habilitar delegação automática.

Fase 3 — Orquestração

- Configurar o agente principal para classificar tarefas e delegar subtarefas delimitadas.
- Definir regras explícitas para o uso de modelo pago e local.
- Executar um conjunto de prompts de teste que cubra tarefas simples, complexas e ambíguas.

Fase 4 — Fallback robusto

- Testar quota esgotada, rate limit, timeout, erro de autenticação e modelo local indisponível.
- Garantir que falhas de quota parem retries repetidos e acionem a rota local.
- Verificar que o estado da tarefa e as alterações em arquivos sejam preservados.

Fase 5 — Otimização

- Medir consumo de tokens, tempo, memória e taxa de sucesso.
- Ajustar contexto e concorrência com base em medições.
- Adicionar novos agentes apenas quando houver ganho comprovado.

Checklist de aceitação

TESTE	CRITÉRIO DE ACEITAÇÃO
Tarefa simples	É concluída localmente sem chamar o provedor pago.
Tarefa complexa com cota disponível	Usa o modelo pago quando a política assim determina.
Cota paga esgotada	Fallback local automático, sem loop de tentativas pagas.
Modelo local indisponível	Erro claro, progresso preservado e nenhuma falsa declaração de sucesso.
Código alterado	Diff revisado e testes relevantes executados ou explicitamente marcados como não executados.

7. Decisão recomendada

Comece com seis agentes cadastrados, um modelo local de 3B, uma chamada local simultânea e um modelo pago principal. Configure o fallback para ativar o modo local após erro de quota e mantenha-o até o término da tarefa. Só aumente modelos e paralelismo depois de medir os resultados.

DECISÃO	RECOMENDAÇÃO INICIAL
Interface	OpenCode CLI, sem IDE obrigatória.
Inferência local	Ollama com Qwen2.5-Coder 3B como primeira avaliação.
Agentes	Explorer, implementação Laravel, testes, revisão, documentação e orquestrador.
Fallback	Automático para quota esgotada, rate limit persistente e indisponibilidade; sem retry infinito.
Concorrência	Uma inferência local por vez inicialmente.
Validação	Testes reais, diff revisado e relatório transparente.

Informações necessárias antes da configuração final

Para gerar os arquivos finais sem incompatibilidades, confirmar: (1) sistema operacional — Ubuntu, Windows 11 ou WSL2; (2) versão exata do OpenCode; (3) provedor pago principal; (4) modelo local e nome do modelo conforme aparece no Ollama/OpenCode.

Referências oficiais para validação

OpenCode — documentação de agentes: https://opencode.ai/docs/agents/
OpenCode — documentação de configuração: https://opencode.ai/docs/config/
Ollama — biblioteca de modelos: https://ollama.com/library

> Observação de manutenção: nomes de modelos, formatos de configuração e mecanismos de fallback podem mudar. Este documento descreve a arquitetura e os critérios de implementação; valide a sintaxe contra a versão instalada antes de aplicar arquivos em um projeto real.